

Unit 08: Software Testing

CONTENTS

Objective

Introduction

8.1 Software Testing

8.2 Fundamentals of Testing

8.3 Testing Objectives

8.4 Test Design

8.5 Test Plan

8.6 Test Plan as per IEEE 829 Standard

8.7 Test Cases

8.8 Test Case Template

Summery

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objective

After studying this unit, you will be able to:

- Discuss the fundamentals of Testing
- Test design and plan
- Test case template

Introduction

Testing is the operation of the software with real or simulated inputs to demonstrate that a product satisfies its requirements and, if it does not, to identify the specific differences between expected and actual results. Software Testing is evaluation of the software against requirements gathered from users and system specifications.

Testing a program consists of providing the program with a set of test inputs (or test cases) and observing if the program behaves as expected. If the program fails to behave as expected, then the conditions under which failure occurs are noted for later debugging and correction.

Some commonly used terms associated with testing are:

Failure: This is a manifestation of an error (or defect or bug). But, the mere presence of an error may not necessarily lead to a failure.

Test case: This is the triplet [I,S,O], where I is the data input to the system, S is the state of the system at which the data is input, and O is the expected output of the system.

Test suite: This is the set of all test cases with which a given software product is to be tested.

Aim of Testing

The aim of the testing process is to identify all defects existing in a software product. However for most practical systems, even after satisfactorily carrying out the testing phase, it is not possible to guarantee that the software is error free. This is because of the fact that the input data domain of most software products is very large. It is not practical to test the software exhaustively with respect to each value that the input data may assume. Even with this practical limitation of the testing process, the importance of testing should not be underestimated. It must be remembered that testing does expose many defects existing in a software product. Thus, testing provides a practical way of reducing defects in a system and increasing the users' confidence in a developed system.

8.1 Software Testing

Software Testing is the process of executing a program or system with the intent of finding errors. Or, it involves any activity aimed at evaluating an attribute or capability of a program or system and determining that it meets its required results. Software is not unlike other physical processes where inputs are received and outputs are produced. Where software differs is in the manner in which it fails. Most physical systems fail in a fixed (and reasonably small) set of ways.

By contrast, software can fail in many bizarre ways. Detecting all of the different failure modes for software is generally infeasible.

Unlike most physical systems, most of the defects in software are design errors, not manufacturing defects. Software does not suffer from corrosion, wear-and-tear – generally it will not change until upgrades, or until obsolescence. So once the software is shipped, the design defects – or bugs – will be buried in and remain latent until activation.

Software bugs will almost always exist in any software module with moderate size: not because programmers are careless or irresponsible, but because the complexity of software is generally intractable – and humans have only limited ability to manage complexity. It is also true that for any complex systems, design defects can never be completely ruled out.

Discovering the design defects in software is equally difficult, for the same reason of complexity. Because software and any digital systems are not continuous, testing boundary values are not sufficient to guarantee correctness. All the possible values need to be tested and verified, but complete testing is infeasible. Exhaustively testing a simple program to add only two integer inputs of 32-bits (yielding 2^{64} distinct test cases) would take hundreds of years, even if tests were performed at a rate of thousands per second. Obviously, for a realistic software module, the complexity can be far beyond the example mentioned here. If inputs from the real world are involved, the problem will get worse, because timing and unpredictable environmental effects and human interactions are all possible input parameters under consideration.

A further complication has to do with the dynamic nature of programs. If a failure occurs during preliminary testing and the code is changed, the software may now work for a test case that it didn't work for previously. But its behavior on pre-error test cases that it passed before can no longer be guaranteed. To account for this possibility, testing should be restarted. The expense of doing this is often prohibitive.

Testing is usually performed for the following purposes:

To Improve Quality

As computers and software are used in critical applications, the outcome of a bug can be severe. Bugs can cause huge losses. Bugs in critical systems have caused airplane crashes, allowed space shuttle missions to go awry, halted trading on the stock market, and worse. Bugs can kill. Bugs can cause disasters. The so-called year 2000 (Y2K) bug has given birth to a cottage industry of consultants and programming tools dedicated to making sure the modern world doesn't come to a screeching halt on the first day of the next century. In a computerized embedded world, the quality and reliability of software is a matter of life and death.

Quality means the conformance to the specified design requirement. Being correct, the minimum requirement of quality, means performing as required under specified circumstances. Debugging, a narrow view of software testing, is performed heavily to find out design defects by the programmer. The imperfection of human nature makes it almost impossible to make a moderately complex program correct the first time. Finding the problems and get them fixed, is the purpose of debugging in programming phase.

For Verification & Validation (V&V)

Just as topic Verification and Validation indicated, another important purpose of testing is verification and validation (V&V). Testing can serve as metrics. It is heavily used as a tool in the V&V process. Testers can make claims based on interpretations of the testing results, which either the product works under certain situations, or it does not work. We can also compare the quality among different products under the same specification, based on results from the same test.

We cannot test quality directly, but we can test related factors to make quality visible. Quality has three sets of factors – functionality, engineering, and adaptability. These three sets of factors can be thought of as dimensions in the software quality space. Each dimension may be broken down into its component factors and considerations at successively lower levels of detail. The table below illustrates some of the most frequently cited quality considerations.

8.2 Fundamentals of Testing

Testing software can be considered as the only destructive (psychologically) step in the entire life cycle of software production. Although all the initial activities aimed at building a product, the testing is done to find errors in software.

Benefits of Testing

The following are the benefits of testing:

It reveals the errors in the software.

It ensures that software is functioning as per specifications and it meets all the behavioral requirements as well.

The data obtained during testing is indicative of software reliability and quality as a whole.

It indicates presence of errors and not absence of errors.

Testing Principles

Before coming up with ways to design efficient test cases, one must understand the basic principles of testing:

- ***Test Cases must be Traceable to Requirements***: Because software testing reveals errors, so, the severe defects will be those that prevent the program from acting as per the customer's expectations and requirements.
- ***Test Planning must be done before Beginning Testing***: Test planning can begin soon after the requirements specification is complete and the detailed test cases can be developed after the design has been fixed.
- ***Pareto Principle Applies to Software Testing***: Pareto principle states that 80 percent of the uncovered errors during testing will likely be traceable to 20 percent of all the program components. Thus, the main aim is to thoroughly test these 20 percent components after identifying them.
- ***Testing should begin with Small and End in Large***: The initial tests planned and carried out are usually individual components and as testing progresses the aim shifts to find errors in integrated components of the system as whole rather than individual components.
- ***Exhaustive Testing is Impossible***: For a normal sized program the number of permutations of execution paths is very huge. Thus, it is impossible to execute all the combinations possible. Thus, while designing a test case it should be kept in minds that it must cover the maximum logic and the component-level design.
- ***Efficient Testing can be conducted only by a Third Party***: The highest probability of Notes finding errors exists when the testing is not carried by the party which develops the system.

8.3 Testing Objectives

Various testing objectives are:

- Executing a program in order to find errors.
- A good test case is the one that has a high probability of finding an undiscovered error.
- A successful test is the one that reports an as – yet undiscovered error.

Testability

A program developed should be testable i.e. it should be possible to test the program. The testability of a program can be measured in terms of few properties like: operability, observability, controllability, decomposability, simplicity, stability, understandability, etc.

These characteristics are mentioned here:

- The probability of finding error should be high. In order to achieve this, tester must understand the actual functionality of the software and think of a suitable condition that can lead to failure of the software.
- A test case must be non-redundant. Because the testing time and resources are limited, it will waste a lot of time to conduct the same test again and again. Every test must have a distinct purpose.
- A test case must be of the best quality. In a group of similar test cases, the one that covers the maximum scenarios to uncover maximum errors should only be used.
- A good test case should neither be too simple nor too complex. A complex test that includes several other tests can lead to masking of errors. Thus, each test case should be executed separately.

Error, Fault, Failure

Many people associate quality of a software system with a suggestion that some software troubles are estimated to happen (or not). And in the case if such troubles do occur, the negative influence is estimated to be minimal. Clue to the correctness perspective of software quality is in the concept of failure, fault, and error.

Error

An error is a person act that generates an erroneous result. The term error is used in two different ways. It refers to the discrepancy between a computed, observed, or measured value and the true, specified, or theoretically correct value. That is error refers to the difference between the actual output of software and the correct output. The term error is a discrepancy between a computed, observed, or measured value or condition and the true, specified, or theoretically correct value or condition

Fault

Fault is a condition that causes a system to fail in performing its required function. It refers to an underlying condition within software that causes failure to happen. A fault is the basic reason for software malfunction and is synonymous with the commonly used term bug. It is an incorrect step, process, or data definition in a computer program which causes the program to perform in an unintended or unanticipated manner.

Failure

Failure is the inability of a system or component to perform a required function according to its specifications. It refers to a behavioral deviation from the user wants or the product specification. A software failure occurs if the behavior of the software is different from the specified behavior.

Problems in Testing

- Software Testing is a critical element of Software Quality Assurance
- It represents the ultimate review of the requirements, specification, the design and the code.
- It is the most widely used method to ensure Software Quality.
- For most software, exhaustive testing is not possible.
- Many organizations spend 40-50% of development time in testing.

- Testing is the most expensive way to remove defects during software development.
- Problems in testing are one of the major contributors to cost and scheduled overruns.

8.4 Test Design

Test design is the act of creating and writing test suites for testing a software. The test design revolves around tests themselves, the test conditions and ways that testing will be approached.

Test design involves creating and writing test suites for testing a software, but will require specificity and detailed input. The fundamental challenge of test design is that there are infinitely many different tests that you could run, but there is not enough time to run them all. A subset of tests must be selected; small enough to run, but well-chosen enough that the tests find bug and expose other quality-related information.

Test design is a process that describes “how” testing should be done. It includes processes for the identifying test cases by enumerating steps of the defined test conditions. The testing techniques defined in test strategy or plan is used for enumerating the steps. The test cases may be linked to the test conditions and project objectives directly or indirectly depending upon the methods used for test monitoring, control and traceability. The objectives consist of test objectives, strategic objectives and stakeholder definition of success.

When to create test design?

After the test conditions are defined and sufficient information is available to create the test cases of high or low level, test design for a specified level can be created. For lower level testing, test analysis and design are combined activity. For higher level testing, test analysis is performed first, followed by test design.

There are some activities that routinely take place when the test is implemented. These activities may also be incorporated into the design process when the tests are created in an iterative manner. An example of such a case is creation of test data. Test data will definitely be created during the test implementation. So it is better to incorporate it in the test design itself.

This approach enables optimization of test condition scope by creating low or high level test cases automatically.

Good test design supports

Defining and improving quality related processes and procedures (quality assurance). Evaluating the quality of the product with regards to customer expectations and needs (quality control). Finding defects in the product (software testing).

Prerequisites of test design

- Appropriate specification (test bases)
- Risk and complexity analysis.
- Historical data of your previous developments (if exists).

8.5 Test Plan

A test plan is a detailed document which describes software testing areas and activities. It outlines the test strategy, objectives, test schedule, required resources, test estimation and test deliverables.

A test plan can also include a test strategy, which outlines the testing approach, and gives generic details for teams to follow. Test plan gives specific responsibilities to team members, the test strategy ensures that anyone is able to execute the tests, aligning with agile practices.

Types of Test Plan

- Master Test Plan
- Phase Test Plan
- Testing Type Specific Test Plans

Master Test Plan

Master Test Plan is a type of test plan that has multiple levels of testing. It includes a complete test strategy.

Phase Test Plan

A phase test plan is a type of test plan that addresses any one phase of the testing strategy. For example, a list of tools, a list of test cases, etc.

Specific Test Plans

Specific test plan designed for major types of testing like security testing, load testing, performance testing, etc. In other words, a specific test plan designed for non-functional testing.

8.6 Test Plan as per IEEE 829 Standard

- Analyze the product
- Design the Test Strategy
- Define the Test Objectives
- Define Test Criteria
- Resource Planning
- Plan Test Environment
- Schedule & Estimation
- Determine Test Deliverables

Analyze the product

Who will use the website?

What is it used for?

How will it work?

What are software/ hardware the product uses?

Design the Test Strategy

The project's testing objectives and the means to achieve them

- Determines testing effort and costs
- Define Scope of Testing
- Identify Testing Type
- Document Risk & Issues
- Create Test Logistics

Define Test Objective

Test Objective is the overall goal and achievement of the test execution.

List all the software features (functionality, performance, GUI...) which may need to test.

Define the target or the goal of the test based on above features.

Define Test Criteria

Test Criteria is a standard or rule on which a test procedure or test judgment can be based. There're 2 types of test criteria as following

Suspension Criteria

Exit Criteria

Resource Planning

Resource plan is a detailed summary of all types of resources required to complete project task.

Resource could be human, equipment and materials needed to complete a project

Plan Test Environment

A testing environment is a setup of software and hardware on which the testing team is going to execute test cases.

The test environment consists of real business and user environment, as well as physical environments, such as server, front end running environment.

Schedule and Estimation

In the Test Estimation phase, break out the whole project into small tasks and add the estimation for each task.

Test Deliverables

Test Deliverables is a list of all the documents, tools and other components that has to be developed and maintained in support of the testing effort.

Test Plan Guidelines

- Collapse your test plan.
- Avoid overlapping and redundancy.
- If you think that you do not need a section that is already mentioned above, then delete that section and proceed ahead.
- Be specific. For example, when you specify a software system as the part of the test environment, then mention the software version instead of only name.
- Avoid lengthy paragraphs.
- Use lists and tables wherever possible.
- Update plan when needed.
- Do not use an outdated and unused document.

Importance of Test Plan

- The test plan gives direction to our thinking. This is like a rule book, which must be followed.
- The test plan helps in determining the necessary efforts to validate the quality of the software application under the test.
- The test plan helps those people to understand the test details that are related to the outside like developers, business managers, customers, etc.
- Important aspects like test schedule, test strategy, test scope etc are documented in the test plan so that the management team can review them and reuse them for other similar projects.

8.7 Test Cases

The test case a group of conditions under which a tester determines whether a software application is working as per the customer's requirements or not. A Test Case contains test steps, test data, precondition, post condition developed for specific test scenario to verify any requirement.

A test case is a set of instructions on "HOW" to validate a particular test objective/target, which when followed will tell us if the expected behavior of the system is satisfied or not.

A test case is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly. The process of developing test cases can also help find problems in the requirements or design of an application. Having test cases that are good at revealing the presence of faults is central to successful testing. Ideally, we would like to determine a set of test cases such that successful execution of all of them implies that there are no errors in the program. This ideal goal cannot usually be achieved due to practical and theoretical constraints.

As each test case costs money, effort is needed to generate the test case, machine time is needed to execute the program for that test case, and more effort is needed to evaluate the results. Therefore, we would also like to minimize the number of test cases needed to detect errors.

Software Engineering Practices

These are the two fundamental goals of a practical testing activity – maximize the number of errors detected and minimize the number of test cases. With selecting test cases the primary objectives is to ensure that if there is an error or fault in the program, it is exercised by one of the test cases.

An ideal test case set is one that succeeds (meaning that its execution reveals no errors) only if there are no errors in the program. For this test selection criterion can be used. There are two aspects of test case selection – specifying a criterion for evaluating a set of test cases, and generating a set of test cases that satisfy a given criterion.

There are two fundamental properties for a testing criterion: reliability and validity. A criterion is reliable if all the sets that satisfy the criterion detect the same errors. A criterion is valid if for any error in the program there is some set satisfying the criterion that will reveal the error.

Some axioms capturing some of the desirable properties of test criteria have been proposed. The first axiom is the applicability axiom, which states that for every program there exists a test set T that satisfies the criterion.

This is clearly desirable for a general-purpose criterion: a criterion that can be satisfied only for some types of programs is of limited use in testing. The anti-extensionality axiom states that there are programs P and Q, both of which implement the same specifications, such that a test set T satisfies the criterion for P but does not satisfy the criterion for Q.

The anti-decomposition axiom states that there exists a program P and its component Q such that a test case set T satisfies the criterion for P and T1 is the set of values that variables can assume on entering Q for some test case in T and T1 does not satisfy the criterion for Q. Essentially, the axiom says that just because the criterion is satisfied for the entire program, it does not mean that the criterion has been satisfied for its components.

The anti-composition axiom states that there exist program P and Q such that T satisfies the criterion for P and the outputs of P for T satisfy the criterion for Q, but T does not satisfy the criterion for the parts P and Q does not imply that the criterion has been satisfied by the program comprising P, Q. It is very difficult to get a criterion that satisfies even these axioms. This is largely due to the fact that a program may have paths that are infeasible, and one cannot determine these infeasible paths algorithmically as the problem is undesirable.

Why test cases?

- To require consistency in the test case execution
- To make sure a better test coverage
- It depends on the process rather than on a person
- To avoid training for every new test engineer on the product

To make sure a better test coverage

We should cover all possible scenarios and document it, so that we need not remember all the scenarios again and again.

It depends on the process rather than on a person

A test engineer has tested an application during the first release, second release, and left the company at the time of third release.

If the person is not there for the third release, it becomes difficult for the new person. Hence all the derived values are documented so that it can be used in the future.

To avoid giving training for every new test engineer on the product

When the test engineer leaves, he/she leaves with a lot of knowledge and scenarios.

Those scenarios should be documented so that the new test engineer can test with the given scenarios and also can write the new scenarios.

Best Practice for writing Test Case

- Test Cases need to be simple and transparent
- Create Test Case with End User in Mind
- Avoid test case repetition
- Do not assume

- Ensure 100% Coverage
- Test Cases must be identifiable
- Implement Testing Techniques
- Self-cleaning
- Repeatable and self-standing
- Peer Review.

Test Cases need to be simple and transparent

Create test cases that are as simple as possible. They must be clear and concise as the author of the test case may not execute them.

Use assertive language like go to the home page, enter data, and click on this and so on.

Create Test Case with End User in Mind

The ultimate goal of any software project is to create test cases that meet customer requirements and is easy to use and operate.

A tester must create test cases keeping in mind the end user perspective.

Avoid test case repetition

Do not repeat test cases. If a test case is needed for executing some other test case, call the test case by its test case id in the pre-condition column.

Do not assume

Do not assume functionality and features of your software application while preparing test case. Stick to the Specification Documents.

Ensure 100% Coverage

Make sure you write test cases to check all software requirements mentioned in the specification document.

Use Traceability Matrix to ensure no functions/ conditions is left untested.

Test Cases must be identifiable

Name the test case id such that they are identified easily while tracking defects or identifying a software requirement at a later stage.

Implement Testing Techniques

Boundary Value Analysis (BVA): As the name suggests it's the technique that defines the testing of boundaries for a specified range of values.

Equivalence Partition (EP): This technique partitions the range into equal parts/groups that tend to have the same behavior.

State Transition Technique: This method is used when software behavior changes from one state to another following particular action.

Error Guessing Technique: This is guessing/anticipating the error that may arise while doing manual testing. This is not a formal method and takes advantages of a tester's experience with the application.

Self-cleaning: The test case you create must return the Test Environment to the pre-test state and should not render the test environment unusable. This is especially true for configuration testing.

Repeatable and self-standing and Peer Review: The test case should generate the same results every time no matter who tests it. After creating test cases, get them reviewed by your colleagues. Your peers can uncover defects in your test case design, which you may easily miss.

Types of test cases

- Functionality Test Cases
- User Interface Test Cases
- Performance Test Cases

- Integration Test Cases
- Usability Test Cases
- Database Test Cases
- Security Test Cases
- User Acceptance Test Cases

Functionality test cases are used to discover if an application's interface works with the rest of the system and its users. The tests identify the success or failure of functions that the software is expected to perform.

User interface test cases are used to verify that specific pieces of the Graphical User Interface (GUI) look and work as expected. These types of test cases can be used to identify cosmetic inconsistencies, grammar and spelling errors, links, and any other elements the user interacts with or sees.

Performance test cases validate response times and overall effectiveness of an application. After executing an action, how long does it take for the system to respond? Performance test cases should have a very clear set of success criteria.

Integration test cases are meant to determine how different modules interact with each other. The main purpose with integration test cases are to ensure interfaces between the different modules are working properly.

Usability test cases can often be referred to as "tasks" or "scenarios". Rather than providing detailed step-by-step instructions to execute the test, the tester is presented with a high level scenario or task to complete.

Database Test Cases: Test cases for database testing examine what's happening behind the scenes. Database tests are used to verify the developer has written the code in a way that stores and handles data in a consistent, safe manner.

Security test cases help ensure the application restricts actions and permissions wherever necessary. These test cases are written to protect data when and where it needs to be safeguarded.

User acceptance test cases, or "UAT" test cases, help the team test the user acceptance testing environment. These test cases should be broad, covering all areas of the application.

Test case Advantages

- Test cases ensure good test coverage
- Help improve the quality of software,
- Decreases the maintenance and software support costs
- Help verify that the software meets the end user requirements
- Allows the tester to think thoroughly and approach the tests from as many angles as possible
- Test cases are reusable for the future – anyone can reference them and execute the test.

8.8 Test Case Template

A test case template is a document containing an organized list of test cases for different test scenarios that check whether or not the software has the intended functionality.

It is well-designed document for developing and better understanding of the test case data for a particular test case scenario.

Test case template fields

Test case ID	Test Priority
Name of the Module	Test Designed by
Date of test designed	Test Executed by
Date of the Test Execution	Name or Test Title
Description of Test	Pre-condition

Dependencies	Test Steps
Test Data	Expected Results
Post-Condition	Actual Result
Status (Fail/Pass)	Notes

Test case template example

Test Case ID	Test Case Name	Summary	Expected Results	Remarks
<unique id to identify the test case>	<short name to identify the test case>	<brief summary of this test case>	<brief explanation of what the expected results of this test case should be>	<any additional remarks>

Test case template example

Test Case ID	TC_Functionality_01		
Description			
Module			
Prepared By	Qatutorial.com	Date Prepared	
Reviewed / Updated		Date Reviewed	
Tested By	Qatutorial.com	Date Tested	
Test Activities			
Sl. No.	Step Description	Expected Results	Actual Results
Test Data Sets			
Data Type	Data Set 1	Data Set 2	Data Set 3
Test Case Result			

Summery

A high quality software product satisfies user needs, conforms to its requirements, and design specifications and exhibits an absence of errors. Techniques for improving software quality include systematic quality assurance procedures, walkthroughs, inspections, static analysis, unit testing integration testing, acceptance testing etc. Testing plays a critical role in quality assurance for software. Testing is a dynamic method for verification and validation. In it the system is executed and the behavior of the system is observed. Due, to this testing observes the failure of the system,

Software Engineering Practices

from which the presence of faults can be deduced. The goal of the testing is to detect the errors so there are different levels of testing. Unit testing focuses on the errors of a module while integration testing tests the system design.

A test plan outlines the strategy that will be used to test an application, the resources that will be used, the test environment in which testing will be performed, and the limitations of the testing and the schedule of the testing activities.

Keywords

Debugging: Debugging is the activity of locating and correcting errors. It starts once a failure is detected.

ET: Error Tolerance

Functional Testing: Functional testing is also known as black box testing.

Software Testing: Software Testing is the process of executing a program or system with the intent of finding errors.

Test plan

Test cases

Self Assessment

1. Good test design supports____
 - A. Defining and improving quality related processes and procedures
 - B. Evaluating the quality of the product with regards to customer expectations and needs
 - C. Finding defects in the product
 - D. All of above
2. What are the Prerequisites of test design?
 - A. Appropriate specification
 - B. Risk and complexity analysis.
 - C. Historical data of your previous developments
 - D. All of above
3. Manual Testing includes.
 - A. Unit testing
 - B. Smoke testing
 - C. Black box testing
 - D. None of above
4. What are different Types of Software Testing?
 - A. Functional Testing
 - B. Non-Functional Testing
 - C. Maintenance
 - D. All of above
5. Which is not a part of Functional testing?
 - A. Localization
 - B. Globalization

-
- C. Black box testing
 - D. None of above
6. What are the Types of Test Plan?
- A. Master Test Plan
 - B. Phase Test Plan
 - C. Testing Type Specific Test Plans
 - D. All of above
7. As per IEEE 829 standard which is not part of test plan?
- A. Design the Test Strategy
 - B. Feasibility study
 - C. Define the Test Objectives
 - D. Define Test Criteria
8. What are the best practice for writing Test Case?
- A. Create Test Case with End User in Mind
 - B. Avoid test case repetition
 - C. Do not assume
 - D. All of above
9. Equivalence Partition is part of ____
- A. Design
 - B. Testing Technique
 - C. Maintenance
 - D. All of above
10. What are the types of test cases?
- A. User Interface Test Cases
 - B. Performance Test Cases
 - C. Integration Test Cases
 - D. All of above
11. Which is not part of test case?
- A. Usability
 - B. Database
 - C. Design
 - D. Security
12. Which is test case template fields?
- A. Test Priority
 - B. Name of the Module
 - C. Test Designed by

D. All of above

13. Status (Fail/Pass) is part of __

- A. Design
- B. Test case template fields
- C. Maintenance
- D. None of above

14. Test case is _____

- A. Design activity
- B. Feasibility study
- C. Contains test steps, test data, pre-condition, post condition
- D. None of above

15. Why test case required?

- A. To require consistency in the test case execution
- B. To make sure a better test coverage
- C. To avoid training for every new test engineer on the product
- D. All of above

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. D | 3. C | 4. D | 5. C |
| 6. D | 7. B | 8. D | 9. B | 10. D |
| 11. C | 12. D | 13. B | 14. C | 15. D |

Review Questions

1. Software Testing is the process of executing a program or system with the intent of finding errors. Analyze.
2. What do you know about various special system tests? Explain briefly.
3. Discuss the various debugging approaches.
4. Discuss test plan and its significance.
5. Why we need test plans?
6. What are the Best Practice for writing Test Case?
7. Discuss Error Guessing Technique.
8. What are the different Types of test cases?



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, Second Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering a Practioner's Approach, 5th edition, Tata McGraw
- Hill Higher education.

- Sommerville, Software Engineering, Sixth edition, Pearson Education.



- <http://msdn.microsoft.com/en-us/library/ms978235.aspx>
- http://bazman.tripod.com/what_testing
- <https://www.tutorialspoint.com/>
- <https://www.softwaretestinghelp.com/>